

05 — API Specification

Purpose

The purpose of this document is to define the official API architecture of Mosaica.

This document establishes how external clients communicate with the Mosaica application while preserving:

- Domain responsibilities,
- Aggregate boundaries,
- security principles,
- business workflows.

The API layer acts as a controlled interface between external applications and the Domain Model.

API Design Principles

Principle 1 — Domain-Oriented API Design

API endpoints shall represent business capabilities rather than database structures.

The API shall expose meaningful actions and resources from the Mosaica domain.

Principle 2 — Aggregate Boundary Protection

External clients shall interact with Aggregates through their public interfaces.

Internal Aggregate objects shall never be modified directly.

Principle 3 — User Data Isolation

Every personal archive operation shall occur within the authenticated User context.

Users shall only access their own personal data.

Principle 4 — Shared Metadata Separation

Shared Content data and personal Library data shall remain separate API responsibilities.

Principle 5 — Explicit Business Actions

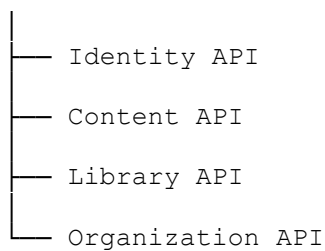
Operations that represent meaningful domain changes shall be expressed explicitly.

The API shall avoid generic update operations that bypass business rules.

API Architecture Overview

Mosaica API consists of four primary areas.

API



1. Identity API

Responsibility

Manages User authentication and identity-related operations.

Scope

Identity API handles:

- registration,
 - authentication,
 - session management,
 - user profile access.
-

Domain Relationship

Identity API provides access context for the User.

It does not manage personal archive business logic.

2. Content API

Responsibility

Provides access to shared cultural metadata.

Aggregate Boundary

Content Aggregate

Responsibilities

Content API manages:

- Content retrieval,
 - Content metadata,
 - contributor information,
 - classification information.
-

Does Not Manage

Content API does not modify:

- Ratings,
- Status,
- Favorites,
- Personal Notes,
- Consumption History.

These belong to Library API.

3. Library API

Responsibility

Manages the User's personal relationship with Content.

Aggregate Boundary

Library Entry Aggregate

Responsibilities

Library API manages:

- creating Library Entries,
 - updating personal evaluation,
 - changing workflow state,
 - managing archive state,
 - recording consumption history,
 - managing personal notes.
-

4. Organization API

Responsibility

Manages User organization features.

Scope

Organization API handles:

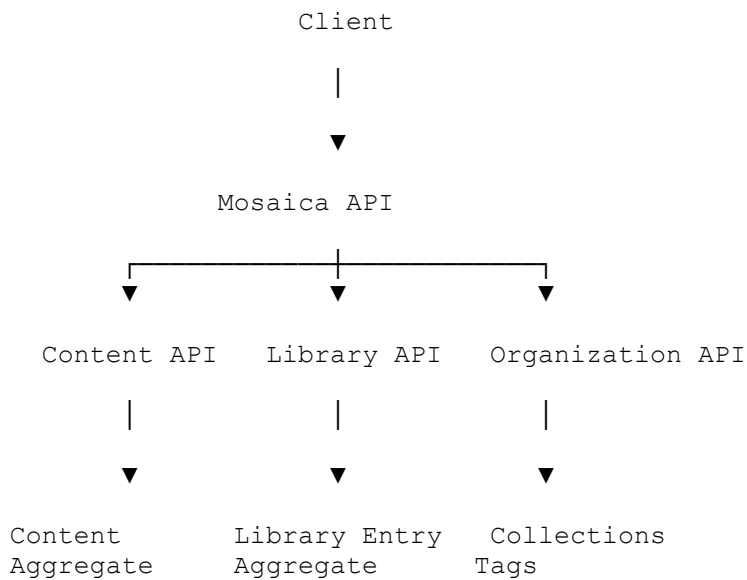
- Collections,
 - Tags.
-

Domain Relationship

Collections and Tags organize Library Entries.

They do not own Library Entries.

API Layer Model



Initial API Resource Model

Content Resources

/contents

Represents shared cultural works.

Library Resources

/library-entries

Represents User-owned archive relationships.

Collection Resources

/collections

Represents User-created organization groups.

Tag Resources

/tags

Represents User-created labels.

Architectural Decisions

Decision 01

API design follows Domain responsibilities rather than database tables.

Decision 02

Library Entry Aggregate is accessed through Library API.

Decision 03

Content Aggregate is accessed through Content API.

Decision 04

Personal and shared data remain separated at API level.

05.01 — API Architecture

Purpose

The purpose of this section is to define the architectural foundation of the Mosaica API layer.

The API Architecture establishes how external clients communicate with the Mosaica system while preserving:

- Domain boundaries,
- Aggregate responsibilities,
- security principles,
- scalability,
- maintainability.

The API layer acts as the controlled communication boundary between external applications and the internal application architecture.

API Architecture Principles

Principle 1 — Domain-Oriented API Design

The API shall represent business capabilities rather than database structures.

Endpoints shall be designed around domain concepts and user actions.

The API shall not expose internal persistence structures directly.

Principle 2 — Aggregate Protection

The API layer shall interact with Domain Aggregates only through their Aggregate Roots.

Internal Entities and Value Objects shall not be modified directly from external requests.

Principle 3 — Separation of Shared and Personal Context

Mosaica contains two different information contexts:

- Shared cultural metadata
- User-owned personal archive data

The API architecture shall preserve this separation.

Principle 4 — Explicit Business Operations

Operations that modify meaningful business state shall be represented as explicit actions.

The API shall avoid generic operations that bypass domain behavior.

Principle 5 — Stateless Communication

The API shall follow a stateless communication model.

Each request shall contain the required authentication context and necessary information to complete the operation.

API Architectural Layers

The Mosaica API architecture consists of the following layers.

Client Application

|

▼

API Layer

|

▼

Application Layer

|

▼

Domain Layer

|

▼

Persistence Layer

Layer Responsibilities

API Layer

Responsibility

The API Layer provides external communication endpoints.

It is responsible for:

- receiving requests,
 - validating request structure,
 - authenticating requests,
 - returning responses.
-

The API Layer Does Not

The API Layer shall not contain:

- business rules,
 - aggregate logic,
 - persistence decisions.
-

Application Layer

Responsibility

The Application Layer coordinates business operations.

It manages:

- use cases,
 - workflow orchestration,
 - communication between aggregates.
-

Examples

Application operations include:

- creating a Library Entry,
 - recording a Consumption Event,
 - updating Content metadata.
-

Domain Layer

Responsibility

The Domain Layer contains the core business model.

It protects:

- entities,
 - value objects,
 - aggregates,
 - business invariants.
-

Examples

Domain responsibilities include:

- valid status transitions,
 - aggregate consistency,
 - business state management.
-

Persistence Layer

Responsibility

The Persistence Layer manages data storage.

It handles:

- repositories,
 - database communication,
 - data retrieval.
-

The Persistence Layer Does Not

The Persistence Layer shall not define business behavior.

API Context Separation

The API exposes different contexts according to domain responsibility.

Mosaica API

Identity Context

Content Context

Library Context

Organization
Context

API Context Responsibilities

Identity Context

Responsible for:

- authentication,
- user identity,
- access context.

Does not manage archive business logic.

Content Context

Responsible for:

- shared cultural metadata,
- Content Aggregate operations.

Does not manage personal User data.

Library Context

Responsible for:

- User archive,
- Library Entry Aggregate operations,
- personal experiences.

Organization Context

Responsible for:

- Collections,
- Tags,
- personal organization features.

API and Aggregate Relationship

The API communicates with Aggregates through Application Services.

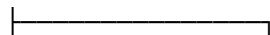
API Request

|



Application Service

|



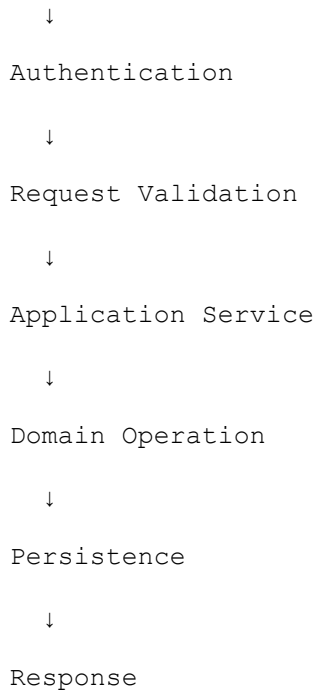
Content
Aggregate

Library Entry
Aggregate

Request Processing Flow

Every API request follows this flow:

Request



API Architecture Decisions

Decision 01

API design follows Domain responsibilities rather than database structures.

Decision 02

Aggregate Roots are the only domain entry points for business operations.

Decision 03

Application Services coordinate operations between API and Domain layers.

Decision 04

Shared metadata and personal archive data remain separated at API level.

Decision 05

Business rules are not implemented inside API controllers.

API Architecture Summary

External Client

↓

API Layer

↓

Application Layer

↓

Domain Layer

↓

Persistence Layer

05.02 — Endpoint Design

Purpose

The purpose of this section is to define the official API endpoint structure of Mosaica.

Endpoints are designed according to Domain responsibilities and Aggregate boundaries.

The API shall expose meaningful business capabilities rather than database operations.

Endpoint Design Principles

Principle 1 — Aggregate-Oriented Endpoints

API operations shall interact with Aggregate Roots.

Internal Aggregate components shall not have independent public endpoints.

Principle 2 — Business Actions Over Field Updates

Complex domain changes shall be represented as business actions.

Generic update operations shall not bypass domain rules.

Principle 3 — Resource Separation

Shared metadata and User-owned information shall remain separate API resources.

Principle 4 — User Context

Personal archive endpoints shall always operate within the authenticated User context.

API Endpoint Structure

The Mosaica API is divided into four main endpoint groups.

API

/auth

/contents

/library-entries

/organization
 /collections
 /tags

1. Identity Endpoints

Base Path

/auth

Register User

Endpoint

POST /auth/register

Purpose

Creates a new User identity.

Login

Endpoint

POST /auth/login

Purpose

Authenticates the User.

Current User

Endpoint

GET /auth/me

Purpose

Returns authenticated User information.

2. Content Endpoints

Base Path

/contents

Get Content List

Endpoint

GET /contents

Purpose

Retrieves shared cultural works.

Supported Operations

- pagination,
 - filtering,
 - searching,
 - sorting.
-

Get Content Detail

Endpoint

GET /contents/{contentId}

Purpose

Retrieves detailed Content information.

Create Content

Endpoint

POST /contents

Purpose

Creates a new Content Aggregate.

Update Content Metadata

Endpoint

PATCH /contents/{contentId}

Purpose

Updates shared Content information.

Architectural Rule

Content endpoints cannot modify:

- Ratings,
 - Status,
 - Favorites,
 - Personal Notes,
 - Consumption History.
-

3. Library Entry Endpoints

Base Path

`/library-entries`

Create Library Entry

Endpoint

POST `/library-entries`

Purpose

Creates a personal relationship between User and Content.

Request Concept

User
+
Content
=
Library Entry

Get User Library

Endpoint

GET `/library-entries`

Purpose

Retrieves the User's personal archive.

Supported Filtering

Examples:

- Status
 - Archive Location
 - Favorite
 - Media Type
 - Rating
-

Get Library Entry Detail

Endpoint

GET /library-entries/{entryId}

Purpose

Retrieves complete personal experience information.

Includes:

- evaluation,
 - workflow,
 - archive state,
 - history,
 - personal memory.
-

Update Evaluation

Rating

Endpoint

PATCH /library-entries/{entryId}/rating

Purpose:

Updates personal rating.

Appreciation Level

Endpoint

PATCH /library-entries/{entryId}/appreciation

Purpose:

Updates personal appreciation.

Update Workflow

Status Change

Endpoint

PATCH /library-entries/{entryId}/status

Purpose:

Changes User progress state.

Update Archive State

Archive Location

Endpoint

PATCH /library-entries/{entryId}/location

Purpose:

Moves Library Entry within archive organization.

Favorite

Endpoint

PATCH /library-entries/{entryId}/favorite

Purpose:

Changes personal favorite state.

Consumption History Endpoints

Record Consumption

Endpoint

POST /library-entries/{entryId}/consumption-events

Purpose:

Records a new User experience.

Get Consumption History

Endpoint

GET /library-entries/{entryId}/consumption-events

Purpose:

Retrieves historical experiences.

Personal Memory Endpoints

Update Personal Note

Endpoint

PATCH /library-entries/{entryId}/note

Purpose:

Updates User reflection.

4. Organization Endpoints

Collections

Base Path:

/collections

Create Collection

POST /collections

Get Collections

GET /collections

Add Library Entry to Collection

POST /collections/{collectionId}/entries/{entryId}

Remove Library Entry from Collection

DELETE /collections/{collectionId}/entries/{entryId}

Tags

Base Path:

/tags

Create Tag

POST /tags

Add Tag to Library Entry

POST /library-entries/{entryId}/tags/{tagId}

Remove Tag

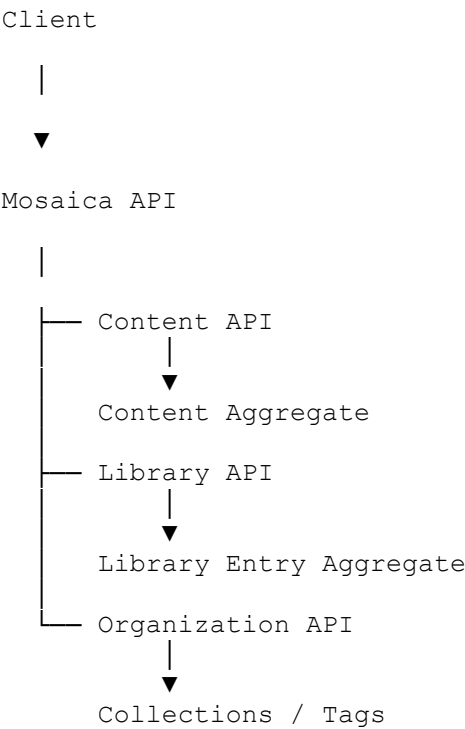
DELETE /library-entries/{entryId}/tags/{tagId}

Endpoint Responsibility Map

Endpoint Group Aggregate Responsibility

/contents	Content Aggregate
/library-entries	Library Entry Aggregate
/collections	Organization Entity
/tags	Organization Entity
/auth	Identity Context

API Boundary Model



Architectural Decisions Confirmed

Decision 01

API endpoints follow Aggregate boundaries.

Decision 02

Value Objects do not receive independent API resources.

Decision 03

Business actions are represented explicitly.

Decision 04

Consumption Events are managed through Library Entry.

Decision 05

Content and personal archive APIs remain separated.

05.03 — Request & Response Models

Purpose

The purpose of this section is to define the official data contracts used between external clients and the Mosaica API.

This section establishes:

- request structures,
- response structures,
- data exposure rules,
- DTO boundaries.

API models shall represent client communication needs while preserving Domain encapsulation.

API Model Principles

Principle 1 — DTO Separation

API Request and Response models shall remain separate from Domain Entities.

Domain objects shall never be exposed directly.

Principle 2 — Purpose-Based Models

Each API model shall represent a specific communication purpose.

A model designed for creation does not need to be identical to a model designed for retrieval.

Principle 3 — Aggregate Protection

API responses shall expose Aggregate information without exposing internal implementation details.

Principle 4 — Minimal Data Exposure

Responses shall contain only information required by the consuming client.

Request Model Structure

Requests represent commands sent by external clients.

General structure:

Client

↓

API Request DTO

↓

Application Service

↓

Response Model Structure

Responses represent information returned to external clients.

General structure:

Domain Result

↓

Response DTO

↓

Client

Content Models

Content Summary Response

Purpose:

Used when listing multiple Content items.

Example:

```
{
  "id": "content-id",
  "title": "Example Title",
  "mediaType": "Movie",
  "releaseYear": 2025
}
```

Content Detail Response

Purpose:

Used when viewing complete Content information.

Contains:

- identity,
- title information,
- media type,

- contributors,
- classifications.

Does not contain:

- User rating,
- Favorite state,
- Personal notes.

Create Content Request

Purpose:

Creates a new Content Aggregate.

Contains:

- title,
- media type,
- metadata information.

Does not contain:

- Library Entry data,
- User preferences.

Library Entry Models

Library Entry represents the most important personal API context.

Library Entry Summary Response

Purpose:

Used for archive listings.

Contains:

- entry identity,
- Content reference,
- title information,
- status,

- rating,
 - favorite state.
-

Library Entry Detail Response

Purpose:

Displays complete personal relationship information.

Contains:

Content Information

- Content identity
- Title
- Media Type

Personal State

- Rating
- Appreciation Level
- Status
- Archive Location
- Favorite

Personal Memory

- Personal Note

History

- Consumption Events
-

Create Library Entry Request

Purpose:

Creates a relationship between User and Content.

Structure:

```
{  
  "contentId": "content-id"  
}
```

Architectural Rule

The client does not provide:

- User identity,
- Aggregate state,
- Created dates.

These are determined by the application context.

Status Update Request

Purpose:

Changes Library Entry workflow state.

Example:

```
{  
  "status": "Completed"  
}
```

Domain Responsibility

The API accepts the requested change.

The Domain determines whether the transition is valid.

Rating Update Request

Example:

```
{  
  "rating": 9  
}
```

Domain Responsibility

The Domain validates rating rules.

Consumption Event Request

Purpose:

Records a new experience.

Example:

```
{  
  "date": "2026-08-01",  
  "note": "Second viewing"  
}
```

Important Rule

Consumption Events are created through Library Entry.

They are not independent API resources.

Collection Models

Collection Response

Contains:

- collection identity,
 - name,
 - entry count.
-

Create Collection Request

Example:

```
{  
  "name": "My Favorites"  
}
```

Add Entry To Collection Request

The relationship is created through an explicit operation.

Example:

POST

/collections/{collectionId}/entries/{entryId}

Tag Models

Tag Response

Contains:

- tag identity,
 - tag name.
-

Create Tag Request

Example:

```
{  
  "name": "Sci-Fi"  
}
```

Authentication Models

Authentication Response

Contains:

- authentication result,
- access information.

The exact security implementation is deferred to Authentication & Authorization design.

Model Exposure Rules

The following rules are established.

Rule 1

Domain Entities are never returned directly.

Rule 2

Aggregate internal structures are not exposed.

Rule 3

Value Objects are represented through API models according to client requirements.

Rule 4

Database fields without business meaning are not exposed.

API Model Relationship

`id="1j9f8a"`
Domain Layer

Content Aggregate
Library Entry Aggregate

↓

Application Layer

↓

API DTO Models

↓

Client

Architectural Decisions Confirmed

Decision 01

API DTO models are separate from Domain Models.

Decision 02

Requests represent client intentions, not database updates.

Decision 03

Responses expose business information, not internal structure.

Decision 04

Aggregate boundaries are preserved through DTO design.

05.04 — Authentication & Authorization

Purpose

The purpose of this section is to define the authentication and authorization architecture of the Mosaica API.

This section establishes:

- user identity verification,
- access control principles,
- resource ownership protection,
- permission boundaries.

The objective is to ensure that Users can securely access their own personal archive while interacting with shared metadata.

Authentication and Authorization Principles

Principle 1 — Authentication Before Personal Access

Every personal archive operation requires an authenticated User context.

A User must be identified before accessing User-owned resources.

Principle 2 — Authorization Based on Ownership

Access to personal data shall be determined by ownership relationships.

A User may only access and modify resources belonging to that User.

Principle 3 — Shared Metadata Accessibility

Shared metadata may be accessed independently from personal ownership.

Content information does not belong to a single User.

Principle 4 — Aggregate Protection

Authorization checks shall occur before Aggregate operations.

A User shall never modify another User's Aggregate.

Authentication Architecture

Authentication Responsibility

Authentication is responsible for answering:

"Who is making this request?"

The authentication system provides:

- User identity,
 - session context,
 - request authorization information.
-

Authentication Flow

User

↓

Login Request

↓

Authentication Service

↓

Identity Verification

↓

Authentication Token

↓

Authenticated API Request

User Context

Every authenticated request contains the current User context.

Example:

Request

User ID:
12345

Action:
Get Library Entries

The application layer uses this context to determine access rights.

Authorization Architecture

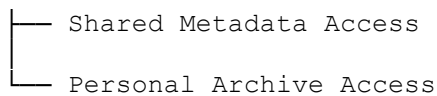
Authorization answers:

"Is this User allowed to perform this action?"

Authorization Categories

Mosaica contains two primary authorization contexts.

Authorization



Shared Metadata Authorization

Resources

Examples:

- Content
- Person
- Genre
- Series
- Universe

Access Model

Shared metadata is not owned by individual Users.

Access rules are therefore based on system permissions rather than ownership.

Example

A User may:

`GET /contents/{id}`

without owning that Content.

Personal Archive Authorization

Resources

Examples:

- Library Entry
- Collection
- Tag
- Consumption History
- Personal Notes

Access Model

These resources are protected by ownership.

Example

A User may:

```
GET /library-entries/{id}
```

only if:

```
library_entry.user_id == authenticated_user.id
```

Ownership Rules

Library Entry

Owner:

User

Access:

Only the owning User.

Collection

Owner:

User

Access:

Only the owning User.

Tag

Owner:

User

Access:

Only the owning User.

Consumption Event

Owner:

Inherited from Library Entry.

Access:

Only through the owning Library Entry Aggregate.

Aggregate Authorization Flow

API Request

↓

Authentication

↓

Identify User

↓

Authorization Check

↓

Aggregate Operation

↓

Persistence

Authorization Responsibilities

API Layer

Responsible for:

- receiving authentication context,
 - rejecting unauthenticated requests.
-

Application Layer

Responsible for:

- ownership verification,
 - permission checks,
 - operation coordination.
-

Domain Layer

Responsible for:

- business invariants,
 - valid state transitions.
-

Security Boundaries

The following boundaries are established.

Boundary 1

A User cannot access another User's Library Entry.

Boundary 2

A User cannot modify another User's Collections or Tags.

Boundary 3

A User cannot modify shared Content without appropriate permission.

Boundary 4

Personal information shall never be exposed through shared metadata endpoints.

Example Authorization Scenarios

Scenario 1 — User Views Own Library

Request:

```
GET /library-entries
```

Result:

Allowed.

Reason:

The request operates within authenticated User context.

Scenario 2 — User Accesses Another User's Entry

Request:

```
GET /library-entries/{otherUserId}
```

Result:

Denied.

Reason:

Ownership validation fails.

Scenario 3 — User Views Content

Request:

`GET /contents/{contentId}`

Result:

Allowed.

Reason:

Content belongs to shared metadata.

Scenario 4 — User Adds Personal Rating

Request:

`PATCH /library-entries/{id}/rating`

Result:

Allowed only if the User owns the Library Entry.

Architectural Decisions Confirmed

Decision 01

Authentication establishes User identity.

Decision 02

Authorization protects User-owned Aggregates.

Decision 03

Shared metadata and personal archive data use different access rules.

Decision 04

Ownership is the primary authorization mechanism for personal resources.

Decision 05

Authorization occurs before Aggregate modification.

05.05 — API Error Handling

Purpose

The purpose of this section is to define the official error handling architecture of the Mosaica API.

This section establishes how API errors are:

- classified,
- represented,
- communicated,
- handled by clients.

The objective is to provide predictable error behavior while preserving the separation between API, Application and Domain responsibilities.

Error Handling Principles

Principle 1 — Consistent Error Structure

All API errors shall follow a common response format.

Clients shall not need separate handling logic for every endpoint.

Principle 2 — Meaningful Error Communication

Errors shall communicate the reason an operation failed.

Technical implementation details shall not be exposed.

Principle 3 — Layer Responsibility Separation

Each architectural layer is responsible for different error categories.

Principle 4 — Security Through Information Control

Error responses shall not reveal sensitive internal information.

Database details, internal stack information and implementation details shall never be exposed.

Error Response Model

All API errors shall follow a standardized structure.

Example:

```
{
  "errorCode": "RESOURCE_NOT_FOUND",
  "message": "The requested resource could not be found.",
  "details": null,
  "timestamp": "2026-08-01T12:00:00Z"
}
```

Error Response Fields

errorCode

A machine-readable identifier used by clients.

Examples:

RESOURCE_NOT_FOUND

VALIDATION_FAILED

UNAUTHORIZED

FORBIDDEN

message

A human-readable explanation.

Messages should explain the problem without exposing internal details.

details

Optional additional information.

Used for validation errors.

timestamp

Records when the error occurred.

Error Categories

Mosaica API errors are divided into five main categories.

API Errors

- Authentication Errors
 - Authorization Errors
 - Validation Errors
 - Domain Errors
 - System Errors
-

1. Authentication Errors

Purpose

Represents failures related to user identity verification.

Examples

- Missing authentication token
 - Invalid token
 - Expired session
-

HTTP Representation

401 Unauthorized

Example

```
{
  "errorCode": "UNAUTHENTICATED",
  "message": "Authentication is required."
}
```

2. Authorization Errors

Purpose

Represents cases where the User is authenticated but lacks permission.

Examples

- Accessing another User's Library Entry
 - Modifying another User's Collection
-

HTTP Representation

403 Forbidden

Example

```
{
  "errorCode": "ACCESS_DENIED",
  "message": "You do not have permission to perform this action."
}
```

3. Validation Errors

Purpose

Represents invalid request data.

Examples

- Missing required fields
 - Invalid format
 - Invalid input value
-

HTTP Representation

400 Bad Request

Example

```
{
  "errorCode": "VALIDATION_FAILED",
  "message": "The request contains invalid data.",
  "details": [
    {
      "field": "rating",
      "message": "Invalid rating value."
    }
  ]
}
```

4. Domain Errors

Purpose

Represents violations of business rules.

Examples

- Invalid Status transition
 - Attempting an unsupported business operation
 - Creating an invalid Library Entry state
-

HTTP Representation

422 Unprocessable Entity

Example

```
{
  "errorCode": "INVALID_STATUS_TRANSITION",
  "message": "The requested status change is not allowed."
}
```

5. System Errors

Purpose

Represents unexpected technical failures.

Examples

- Database connection failure
 - External service failure
 - Internal application error
-

HTTP Representation

500 Internal Server Error

Example

```
{
  "errorCode": "INTERNAL_ERROR",
  "message": "An unexpected error occurred."
}
```

}

Layer Error Responsibilities

API Layer

Responsible for:

- HTTP response formatting,
 - error serialization,
 - client communication.
-

Application Layer

Responsible for:

- operation-level failures,
 - workflow errors,
 - authorization coordination.
-

Domain Layer

Responsible for:

- business rule violations,
 - invalid domain states.
-

Persistence Layer

Responsible for:

- storage failures,
 - infrastructure errors.
-

Domain Error Examples

Example 1 — Invalid Status Change

Operation:

Change Library Entry Status

Problem:

Business rules reject the transition.

Response:

422 Unprocessable Entity

Example 2 — Duplicate Collection Name

Operation:

Create Collection

Problem:

User already has a Collection with the same name.

Response:

422 Unprocessable Entity

Example 3 — Missing Content

Operation:

Create Library Entry

Problem:

Referenced Content does not exist.

Response:

404 Not Found

Error Logging Principles

The API shall separate:

Client Response

Simple and safe information.

Internal Logging

Detailed technical information for developers.

Internal logs may contain:

- stack traces,
- database errors,
- request context.

These details shall never be returned to clients.

Error Handling Flow

Request

↓

API Validation

↓

Application Operation

↓

Domain Validation

↓

Persistence

↓

Response

If an error occurs:

Failure

↓

Error Classification

↓

Standard Error Response

↓

Client

Architectural Decisions Confirmed

Decision 01

All API errors use a common response format.

Decision 02

Domain errors are separated from technical errors.

Decision 03

Business rule failures are represented independently from validation failures.

Decision 04

Internal system information is never exposed through API responses.

Decision 05

HTTP status codes represent error categories consistently.

05.06 — API Versioning

Purpose

The purpose of this section is to define the official versioning strategy of the Mosaica API.

This section establishes how API changes are managed over time while preserving compatibility, stability and predictable client communication.

The objective is to allow future evolution of the API without unnecessary disruption to existing consumers.

API Versioning Principles

Principle 1 — Controlled Evolution

The API shall evolve through controlled and documented changes.

Changes shall not be introduced without considering existing consumers.

Principle 2 — Breaking Changes Require Versioning

Changes that break existing client expectations shall require a new API version.

Principle 3 — Backward Compatibility

Non-breaking improvements should preserve compatibility with existing API consumers.

Principle 4 — Domain Stability

API versions shall represent external communication changes.

They shall not be created for internal implementation changes.

Versioning Strategy

Decision

Mosaica shall use URL-based API versioning.

Format

Example:

```
/api/v1/contents
```

```
/api/v1/library-entries
```

Rationale

URL-based versioning provides:

- clear visibility,
 - simple client understanding,
 - explicit API contracts,
 - easy documentation management.
-

Version Lifecycle

Each API version follows a controlled lifecycle.

Development

↓

Release

↓

Maintenance

↓

Deprecation

↓

Removal

Version States

Active

The current recommended API version.

Supported for new development.

Maintenance

Still supported but no longer receiving major feature expansion.

Deprecated

Marked for future removal.

Clients are encouraged to migrate.

Removed

No longer available.

Breaking Changes

The following changes require a new API version.

Examples:

Response Structure Changes

Before:

```
{  
  "title": "Example"  
}
```

After:

```
{  
  "name": "Example"  
}
```

This breaks existing clients.

Removing Existing Fields

Removing a previously available response field requires version consideration.

Changing Endpoint Behavior

Changing the meaning of an existing operation requires a new version.

Changing Authentication Requirements

Major security model changes require version management.

Non-Breaking Changes

The following changes do not require a new version.

Examples:

Adding Optional Response Fields

Example:

Before:

```
{  
  "title": "Example"  
}
```

After:

```
{  
  "title": "Example",  
  "releaseYear": 2025  
}
```

Adding New Endpoints

Example:

`POST /api/v1/library-entries/{id}/consumption-events`

New functionality can be introduced without breaking existing clients.

Internal Implementation Changes

The following do not require API version changes:

- database optimization,
 - service refactoring,
 - internal architecture improvements.
-

API Documentation Strategy

Each API version shall maintain independent documentation.

Example:

API Documentation

/v1

Contents API
Library API
Organization API

/v2

Contents API
Library API
Organization API

Migration Strategy

When a new API version is introduced:

1. New version becomes available.
 2. Existing version remains supported.
 3. Clients receive migration documentation.
 4. Deprecated version receives a removal timeline.
 5. Old version is removed after transition period.
-

Version Ownership

API version changes require architectural review.

A version change shall evaluate:

- Domain impact,
 - client impact,
 - migration complexity,
 - backward compatibility.
-

Architectural Decisions Confirmed

Decision 01

Mosaica uses URL-based API versioning.

Decision 02

Breaking changes require a new API version.

Decision 03

Non-breaking improvements remain within the existing version.

Decision 04

API versions represent external contract changes, not internal implementation changes.

Decision 05

Deprecated versions require a controlled migration process.

API Version Structure

`/api/v1`

`/contents`

`/library-entries`

/collections

/tags

/auth

05.07 — API Architecture Review

Review Objective

The purpose of this Architecture Review is to validate that the Mosaica API Specification accurately represents the approved Domain Model and Database Architecture.

This review confirms that the API layer provides a controlled communication boundary while preserving:

- Domain responsibilities,
 - Aggregate boundaries,
 - security principles,
 - data ownership,
 - long-term maintainability.
-

Scope Reviewed

The following API architecture components have been reviewed:

- API Architecture
 - Endpoint Design
 - Request & Response Models
 - Authentication & Authorization
 - API Error Handling
 - API Versioning
-

Validation Results

Domain Alignment

Status: Approved

The API architecture remains consistent with the approved Domain Model.

API responsibilities correctly reflect domain responsibilities.

No endpoint introduces new business concepts outside the approved architecture.

Aggregate Protection

Status: Approved

API operations correctly interact with Aggregate Roots.

The following boundaries are preserved:

- Content operations → Content Aggregate
- Library operations → Library Entry Aggregate

Internal Value Objects are not exposed as independent resources.

Endpoint Design

Status: Approved

Endpoints represent business capabilities rather than database structures.

The API avoids direct table-oriented operations.

Examples:

Correct:

```
PATCH /library-entries/{id}/status
```

Incorrect:

```
PATCH /statuses/{id}
```

The approved design preserves business meaning.

Request & Response Models

Status: Approved

API DTO models remain separate from Domain Models.

The API does not expose:

- internal Entity structures,
 - database representations,
 - Aggregate implementation details.
-

Shared and Personal Data Separation

Status: Approved

The API correctly separates:

Shared Context

- Content
- Person
- Genre
- Series
- Universe

Personal Context

- Library Entry
- Rating
- Status
- Favorite
- Personal Notes
- Consumption History

No personal information is exposed through shared metadata operations.

Authentication & Authorization

Status: Approved

Access control follows ownership boundaries.

The authorization model correctly protects User-owned resources.

Confirmed rules:

- Users access only their own Library Entries.
- Users manage only their own Collections.
- Users manage only their own Tags.
- Shared Content follows separate access rules.

Error Handling

Status: Approved

API errors are consistently classified.

The architecture correctly separates:

- Authentication errors,
- Authorization errors,
- Validation errors,
- Domain errors,
- System errors.

Business failures are not incorrectly represented as technical failures.

API Versioning

Status: Approved

The versioning strategy provides controlled API evolution.

Confirmed principles:

- Breaking changes require new versions.
 - Non-breaking changes remain within the current version.
 - Internal implementation changes do not require API version changes.
-

Architectural Decisions Confirmed

The following decisions are now official parts of the Mosaica API Architecture.

Decision 01

The API is designed around Domain responsibilities rather than database structures.

Decision 02

Aggregate Roots define the primary API business boundaries.

Decision 03

Library Entry is the central API boundary for personal user experience.

Decision 04

Content remains a separate shared metadata API context.

Decision 05

API DTO models remain separate from Domain Models.

Decision 06

Personal resources require ownership-based authorization.

Decision 07

API errors follow a standardized response structure.

Decision 08

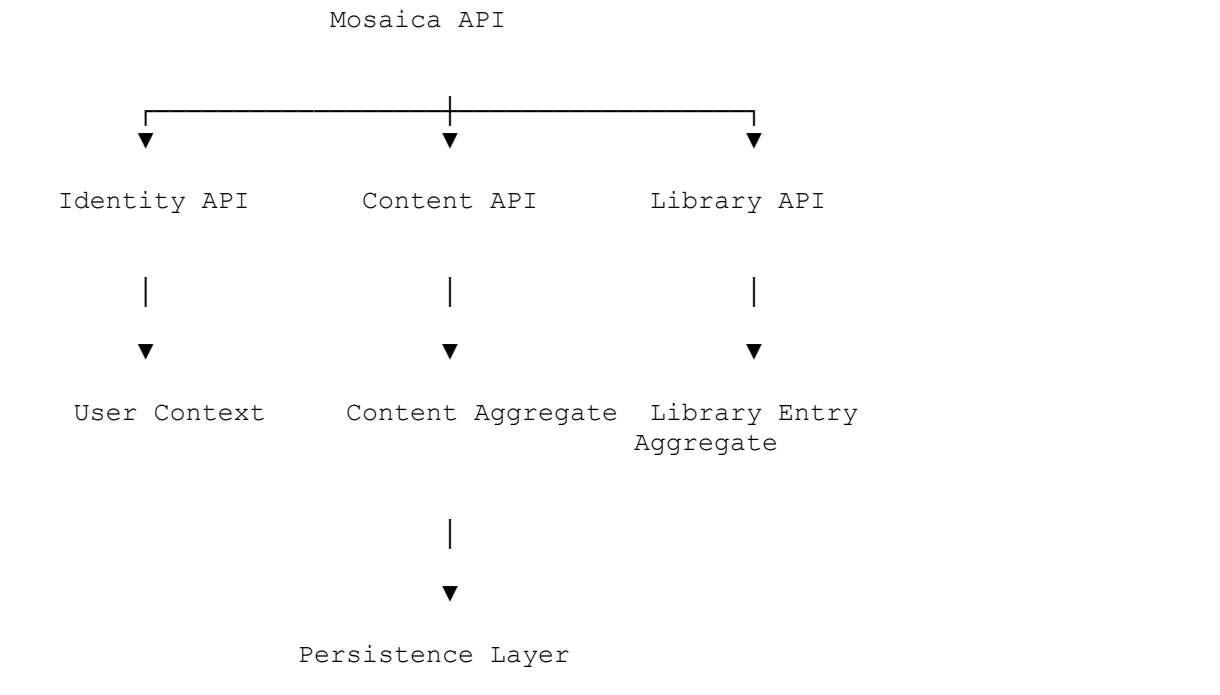
API evolution follows controlled versioning rules.

API Architecture Summary

Client

|





Review Conclusion

The Mosaica API Specification is considered architecturally complete.

The current design provides a stable communication boundary between external clients and the internal application architecture.

The API preserves:

- Domain integrity,
- Aggregate boundaries,
- security requirements,
- data ownership,
- future extensibility.

No blocking architectural issues have been identified.